

CONNECT FOUR MINI AI – MINIMAX

Mini Project Report

Game	Connect Four – 6 × 7 board	AI	Depth-Limited Minimax
Depth	4	Language	Python

1. Problem Formulation

Connect Four is a two-player strategy game played on a **6 × 7 board**. The human uses **X** and the AI uses **O**. Players choose a non-full column and the piece falls to the lowest empty position. The first player to connect four pieces horizontally, vertically, or diagonally wins.

The AI decision problem is represented as a game-tree search. Each board position is a state and each legal column is an action. The AI must consider its possible moves and the opponent's responses. Because the complete game tree is very large, this implementation searches only to **depth 4**.

2. Approach

The project uses **Depth-Limited Minimax**. The AI is the maximizing player and the human is treated as the minimizing player. For every valid AI move, the program simulates the move and recursively explores alternating AI and human turns until a win, draw, or depth limit is reached.

1	Generate valid moves
2	Simulate AI move
3	Explore future states
4	Evaluate at depth 0 / terminal state
5	Propagate MAX/MIN scores
6	Select highest-scoring move

Heuristic Evaluation: AI piece = **+2**, human piece = **-2**, AI center-column piece = **+3 additional**. An AI win receives **+100000** and a human win receives **-100000**. The large terminal values make wins and losses more important than normal positional scores.

3. Complexity

Minimax has approximate time complexity **$O(b^d)$** , where b is the branching factor and d is search depth. Connect Four can have up to seven column choices, so the search grows rapidly as depth increases. Using depth 4 keeps the program manageable for this mini project. Alpha-Beta pruning is not implemented.

4. Results

The game was executed successfully. In one observed AI decision, the program selected **column 4** and reported **2800 Minimax nodes** explored. Automated testing checked board creation, the winning condition, and AI move validity; the observed result was **3 passed**.

Execution / Test	Observed Result
AI game execution	AI chose column 4
Minimax search	2800 nodes explored
Automated testing	3 passed

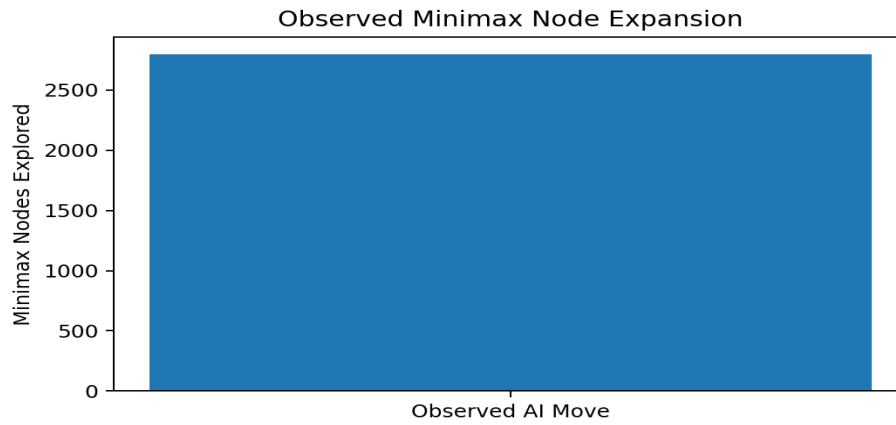


Figure 1. Observed Minimax node expansion for one AI decision.

```
GAME EXECUTION

Problems Output Debug Console Terminal Ports

@maheshhhh7 →/workspaces/connect-four-mini-ai (main) $ python src/connect_four.py
AI thinking...
AI chose 4 | Nodes: 2800
. . . . .
. . . . .
. . . . .
. . . . .
. . . O . .
. . . X . .
1 2 3 4 5 6 7

Your move (1-7):
```

Figure 2. Game execution showing the AI decision and node count.

```
TEST RESULTS

Problems Output Debug Console Terminal Ports

@maheshhhh7 →/workspaces/connect-four-mini-ai (main) $ pip install pytest
@maheshhhh7 →/workspaces/connect-four-mini-ai (main) $ python -m pytest
===== test session starts =====
platform linux -- Python 3.14.2, pytest-9.1.1, pluggy-1.6.0
rootdir: /workspaces/connect-four-mini-ai
plugins: anyio-4.14.2
collected 3 items

tests/test_connect_four.py ... [100%]

===== 3 passed in 0.73s =====
@maheshhhh7 →/workspaces/connect-four-mini-ai (main) $
```

Figure 3. Pytest execution showing 3 passed tests.

5. Reflection

This project provided practical understanding of how an AI agent can make decisions in a two-player game. The main challenge was understanding how simulated board states are generated and how Minimax alternates between maximizing and minimizing decisions.

The depth limit demonstrated the computational cost of game-tree search, while the heuristic allowed the AI to estimate board positions without searching to the end of the game. Pytest testing and node counting provided additional evidence of the implementation's behavior.

The current heuristic is intentionally simple. Future improvements could include Alpha-Beta pruning, stronger two- and three-in-a-row evaluation, explicit threat blocking, and deeper search.

Conclusion

The Connect Four Mini AI successfully demonstrates **Depth-Limited Minimax with heuristic evaluation**. The AI searches possible future states up to depth 4, evaluates positions, and selects a move according to the resulting score. The successful game execution and **3 passed** automated tests provide evidence of the implemented functionality.

Technology: Python | **AI:** Minimax | **Testing:** Pytest | **License:** MIT